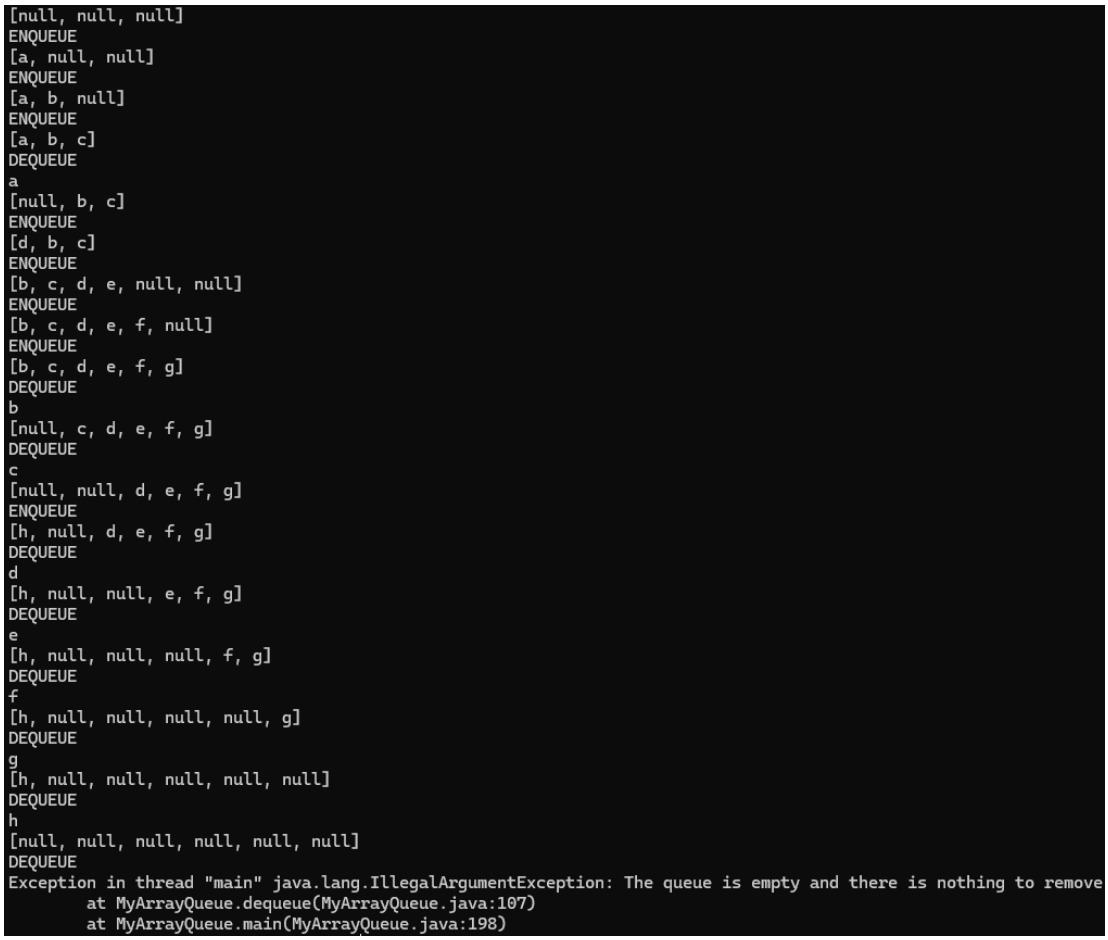
Output example:



With the code written for enqueueing and dequeuing data, our focus was to add and remove elements from the queue and moving the pointer which indicates the last thing added and which item would be removed next. This was to be done within the enqueue function and dequeue functions, of which are called in order at the end of the document. We print enqueue or dequeue, followed by a call to the corresponding function. Whilst doing so, we pass the array to the function, along with the corresponding piece of data that we want to add to the array or remove from the array.

With the creation of this code, we had the aim of optimising speed, such that it takes a very small period of time to perform the algorithm. This was done by simplifying the code into smaller steps which can be easily traced and passed through simply. Building on from this, the code was also developed with the aim of maximising efficiency of the run time and the code itself. I tackled this by developing it in a way which was broken down into two major steps.

Firstly, a check was run against the queue to see whether the queue was viable for the operation, whether that be adding a value or removing one from it. This was done by checking the number of values within the given array and if it was too many to add or too few to remove, it output this. The next step was to add the value required to the part of the array of which the header was pointing at.

To implement the circular queue, it was required that the queue wrap around itself such that once it had ended, it loops back to the start of the queue. This required additional code to be added to calculate the end and start of the queue and when it is appropriate to loop to the start of the queue. This was then tested thoroughly in the main() function in order to ensure security to the problem.

The majority of the code required for this assignment was already given to us, hence it was easier to create the necessary parts to solve the issue presented to us. The algorithm I created was optimal for the assignment provided, requiring not too much code whilst also applying error checking to the problem.